

RELATÓRIO DE AUDITORIA DE SEGURANÇA

reserva-sala-mp (MPGO)

Data da auditoria	30/08/2026
Repositório	reserva-sala-mp (MPGO)
Branch	redesign-v2
Escopo	Código-fonte completo (backend Django/DRF, templates, JS estático, migrations), arquivos de configuração e deploy (settings.py, .env.example, Dockerfile, docker-compose*.yaml, .pre-commit-config.yaml) e histórico do Git.
Categorias avaliadas	(1) Isolamento por dono/organização — equivalente a RLS neste app de organização única; (2) Permissão definida só no navegador; (3) IDOR; (4) Chaves e segredos expostos; (5) Inputs sem tratamento (XSS).

Nota metodológica

Stack detectada: **Django 5.2 + Django REST Framework 3 (DRF)** em Python 3.12, ORM Django ORM (PostgreSQL + extensão pgvector via psycopg2-binary), autenticação django.contrib.auth (sessão + cookie), sem multi-tenant — app de organização única (MPGO), frontend Django Templates server-side + HTMX + Tailwind/DaisyUI (django-tailwind-cli); sem SPA e sem JavaScript autoral além do htmx.min.js vendorizado, deploy via Dockerfile + docker-compose.yml (dev) + docker-compose.prod.yml, gunicorn + WhiteNoise; sem pipeline de CI versionado no repositório. Cada categoria genérica do roteiro de auditoria foi mapeada para o equivalente concreto desta stack antes da varredura — o detalhamento de cada mapeamento está na seção seguinte.

Nota metodológica — mapeamento das categorias para a stack

Este projeto é uma aplicação Django monolítica de **organização única** (o MPGO), sem Supabase e sem separação multi-tenant — por isso as cinco categorias do roteiro padrão de auditoria foram traduzidas para o equivalente real desta stack antes da varredura, como segue.

Banco sem tranca (isolamento)

Este projeto não é multi-tenant: existe uma única organização (MPGO). O mecanismo de isolamento equivalente é o filtro por usuário dono do recurso (`request.user`) em toda consulta de listagem, detalhe, relatório e exportação de reservas — e, para telas administrativas, o filtro por papel (`is_staff`). Não há RLS porque não há Supabase; a auditoria verificou, view por view, se esse filtro por dono/papel existe e é aplicado no servidor.

Permissão definida no navegador

Todas as telas administrativas são Class-Based Views. A auditoria cruzou cada view do `admin_dashboard` e cada endpoint DRF administrativo com a `mixin/permission_class` que o protege no servidor, e não apenas com o que o template esconde.

IDOR

Todo handler que recebe um `pk` (rotas web com `get_object_or_404` e ações de `ViewSet` do DRF) foi lido individualmente para confirmar se o objeto é buscado já filtrado pelo dono, ou se a posse é validada antes de qualquer leitura/escrita sensível.

Chaves expostas

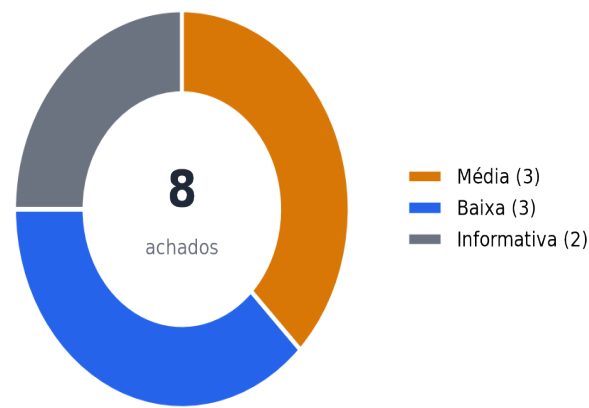
Sem Supabase/Vault: os segredos deste projeto vivem em variáveis de ambiente lidas por `config/settings.py`, no `.env` (não versionado) e nos `docker-compose*.yaml`. A auditoria verificou os defaults de cada variável sensível, se há validação de startup contra valores inseguros, e varreu o histórico do Git e os arquivos versionados por segredos reais.

Inputs sem tratamento (XSS)

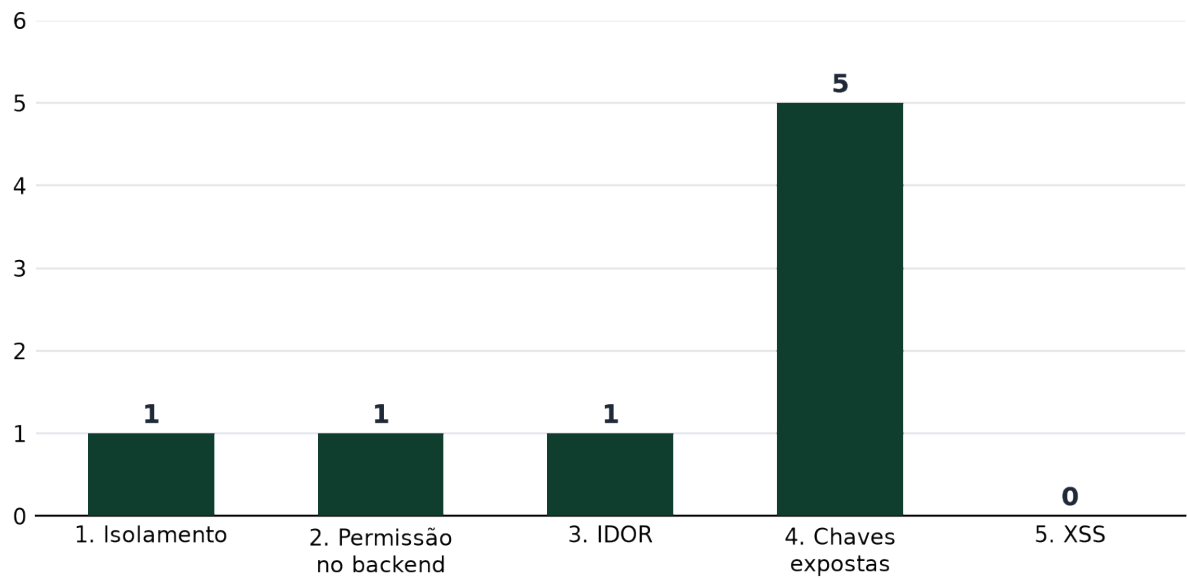
Sem frontend JS autoral (SPA), o vetor equivalente é o autoescape do Django Template Language: uso de `|safe`, `mark_safe`, `format_html` ou `{% autoescape off %}` sobre conteúdo editável por usuário/administrador ou gerado por LLM (o projeto tem dois assistentes de IA). A auditoria varreu todos os templates e o código Python por esses pontos de fuga do autoescape.

Resumo executivo

Severidade	Quantidade
Crítica	0
Alta	0
Média	3
Baixa	3
Informativa	2
Total de achados	8
Pontos fortes verificados	12



Achados por categoria



Não foram identificados achados de severidade crítica ou alta. Os achados de severidade média concentram-se em **defaults e guardas de configuração** (categoria 4) e em uma questão de **modelagem de privilégio** (categoria 2); nenhum deles é uma falha de autorização em produção — todos exigem uma condição operacional adicional (ex.: um operador pular a troca de um placeholder) para serem explorados. A categoria 5 (XSS) não produziu nenhum achado.

Pontos fortes — o que está protegido, com evidência

Lista o que foi verificado no código e está corretamente implementado. Esta seção existe tanto para orientar o time (o que *não* precisa ser mexido) quanto para demonstrar a cobertura desta auditoria — cada item aponta para o arquivo e a linha onde a proteção foi confirmada.

- ✓ **Posse de reserva verificada em duas camadas.** Toda view web que edita uma reserva busca o objeto já filtrado por dono (`get_object_or_404(Reservation, pk=pk, user=request.user)`) — `reservations/views.py:368` (detalhe), `:1085` (edição de detalhes), `:1158` (cancelamento), `1189` (check-in), `:1224/:1250` (reagendamento). Além disso, a camada de serviço repete a checagem e levanta `OwnershipError` (`reservations/services.py:149, 218, 254, 311`), protegendo também as ações do DRF que buscam só por `pk` (ver achado F6, que é apenas um oráculo de existência residual, não um bypass).
- ✓ **`ReservationViewSet.get_queryset` nunca devolve reserva alheia.** `reservations/views.py:102-104` filtra `Reservation.objects.filter(user=self.request.user)` — como as ações padrão do DRF (`retrieve`/`update`/`destroy`) usam esse `queryset` via `get_object()`, um `GET/PATCH/DELETE /api/v1/reservations/` de uma reserva alheia responde 404, não vaza dado nenhum.
- ✓ **Cobertura total de `StaffRequiredMixin` no painel administrativo.** As 26 views de `admin_dashboard/views.py` — espaços, tipos de espaço, atributos, serviços, fila de serviço, política de reservas, calendário geral, relatórios, exportação CSV, reservas, manutenção, usuários e ajuda institucional — herdam de `StaffRequiredMixin` (`admin_dashboard/views.py:48-53`), que testa `request.user.is_staff` no servidor. Não há nenhuma tela cujo controle de acesso dependa só do menu escondido no template.
- ✓ **Endpoints administrativos do DRF exigem `IsAdminUser`.** `MaintenanceBlockViewSet` (`reservations/views.py:88`) e `OccupancyView` (`reservations/views.py:160`) usam `permissions.IsAdminUser`; `SpaceViewSet.get_permissions` (`spaces/views.py:72-76`) restringe `create/update/partial_update/destroy` ao mesmo permission class, deixando leitura para qualquer usuário autenticado.
- ✓ **Nenhum uso de `|safe`, `mark_safe`, `format_html` ou `{% autoescape off %}`.** Varredura completa de `templates/` e de todo o código Python não encontrou nenhuma dessas aberturas do autoescape do Django sobre conteúdo editável por usuário ou administrador.
- ✓ **Texto institucional editável por admin tratado como texto puro, por design documentado.** `core/models.py:43-47` documenta explicitamente a decisão: o corpo de `HelpArticle` "é tratado como texto puro, nunca como HTML" para não abrir "uma porta de injeção na tela mais inocente do sistema"; o template (`templates/core/ajuda.html:84`) imprime cada parágrafo com `{{ paragrafo }}`, auto-escapado.
- ✓ **Resposta dos assistentes de IA (LLM) sempre renderizada com autoescape.** Tanto a busca de salas quanto o assistente documental (RAG) imprimem a resposta do modelo via `{{ result.answer }}` (`templates/knowledge/_assistant_answer.html:24`) sem `|safe` — mesmo que o modelo devolvesse HTML/script no texto, o navegador o exibiria como texto literal.
- ✓ **Upload de imagem de capa validado pelo conteúdo, não pela extensão.** `spaces/validators.py` verifica o formato abrindo o arquivo com Pillow (não confia em Content-Type nem extensão), limita a 5 MB, exclui SVG explicitamente ("XML executável, não bitmap") e `spaces/images.py` reprocesa toda imagem aceita — remove metadados EXIF, achata para WebP e renomeia para um UUID antes de gravar, eliminando o nome escolhido pelo usuário do sistema de arquivos.
- ✓ **`.env` nunca chegou ao controle de versão.** Confirmado via `git log --all --name-only` e `git ls-files: .env` está listado em `.gitignore:54` e nunca foi commitado — só `.env.example` (com placeholders) está versionado.
- ✓ **`docker-compose.prod.yml` recusa subir sem segredos reais.** `SECRET_KEY`, `DATABASE_URL`, `ALLOWED_HOSTS` e `POSTGRES_PASSWORD` usam a sintaxe `${VAR:?mensagem}` (`docker-compose.prod.yml:18-20,41-43`) — o Compose aborta o `up` se a variável não estiver definida, em vez de silenciosamente usar um default inseguro.

-
- ✓ **Nenhum `@csrf_exempt` no projeto; CSRF middleware sempre ativo.** Busca em todo o código Python não encontrou nenhuma view isenta de CSRF; `CsrfViewMiddleware` está na cadeia de middleware (`config/settings.py:74`).
-
- ✓ **Permissão padrão do DRF é `IsAuthenticated`, não `AllowAny`.** `REST_FRAMEWORK.DEFAULT_PERMISSION_CLASSES` (`config/settings.py:209-212`) é `IsAuthenticated` — qualquer endpoint futuro que esqueça de declarar `permission_classes` nasce fechado, não aberto.

Pontos fracos — riscos centrais

Síntese dos riscos identificados, em ordem de severidade. O detalhamento completo — arquivo, linha, trecho de código e explicação de exploração — está na seção "Achados detalhados" a seguir.

Sev.	ID	Achado	Arquivo
Média	F1	Guarda de startup contra SECRET_KEY insegura cobre só o próprio default	config/settings.py:229-234
Média	F3	`seed_data` cria superusuário 'admin' com senha padrão previsível, sem guarda de ambiente	core/seeds/users.py:7-34 e core/management/commands/seed_data.py
Média	F4	Qualquer conta staff pode promover outra conta a staff (admin) — um único tier de privilégio	admin_dashboard/forms.py:16-21, 36-58 e admin_dashboard/views.py (AdminUserCreateView/AdminUserUpdateView)
Baixa	F2	Placeholder de senha do Postgres não é validado, só exigido como não-vazio	.env.example:19 e docker-compose.prod.yml:20
Baixa	F5	API de espaços (DRF) não filtra `is_active`, ao contrário da tela web equivalente	spaces/views.py:47-51, 63-69
Baixa	F6	Oráculo de existência (403 vs. 404) nas ações de reserva da API antes da checagem de posse	reservations/views.py:110-134 (ReservationViewSet.cancel / check_in / reschedule)
Informativa	F7	Nenhum hook de detecção de segredos no pre-commit	.pre-commit-config.yaml
Informativa	F8	Chave real da API Groq presente no `.env` local, lida durante esta auditoria	.env (não versionado)

Achados detalhados por categoria

1. Isolamento por dono/organização

Baixa

F5 — API de espaços (DRF) não filtra `is\_active`, ao contrário da tela web equivalente

Arquivo: `spaces/views.py:47-51, 63-69`

```
class SpaceViewSet(viewsets.ModelViewSet):
    queryset = Space.objects.select_related("space_type").prefetch_related(
        "space_attributes__attribute"
    )
    ...
    fields = ["is_active", "location", "space_type"] # SpaceFilterSet
```

Por que é explorável: `SpaceListView` (a tela web, `spaces/views.py:106`) filtra explicitamente `is\_active=True`. O `SpaceViewSet` do DRF (`/api/v1/spaces/`) não aplica esse filtro por padrão e ainda expõe `is\_active` como filtro exato via `SpaceFilterSet` — qualquer usuário autenticado pode listar/consultar espaços desativados (`GET /api/v1/spaces/?is_active=false`), inclusive salas desligadas por motivo sensível (interditadas, em obra, etc.), que a tela pretende esconder.

Condição de exploração: Requer apenas autenticação (qualquer usuário comum) — não requer papel administrativo. Não cruza organização (app de organização única), mas é uma divergência real entre o controle de acesso da tela e o da API para o mesmo dado.

2. Permissão no navegador vs. backend

Média

F4 — Qualquer conta staff pode promover outra conta a staff (admin) — um único tier de privilégio

Arquivo: `admin_dashboard/forms.py:16-21, 36-58` e `admin_dashboard/views.py` (`AdminUserCreateView/AdminUserUpdateView`)

```
USER_FIELD_WIDGETS = {..., "is_staff": forms.CheckboxInput(...), ...}

class AdminUserCreateForm(forms.ModelForm):
    class Meta:
        fields = ["username", "email", "is_staff", "is_active"]
```

Por que é explorável: `AdminUserCreateView`/`AdminUserUpdateView` (`admin_dashboard/views.py`) são protegidas só por `StaffRequiredMixin`, que testa `request.user.is\_staff` — não `is\_superuser`. O formulário expõe o campo `is\_staff` a qualquer usuário que passe por esse gate. Ou seja: comprometer uma única conta `is\_staff` é suficiente para mintar quantas outras contas staff/admin o atacante quiser, sem nunca precisar de `is\_superuser`.

Condição de exploração: Não é um bypass de autorização (o backend valida `is\_staff` corretamente em toda a rota, cobrindo a categoria 2 do escopo) — é uma observação de modelagem: o sistema não distingue 'administrador de espaços' de 'administrador de contas'. Pode ser intencional para este projeto (uma casa só, um tipo de administrador), mas eleva o raio de impacto de qualquer conta staff comprometida a comprometimento total do sistema.

3. IDOR

Baixa

F6 — Oráculo de existência (403 vs. 404) nas ações de reserva da API antes da checagem de posse

Arquivo: `reservations/views.py:110-134` (`ReservationViewSet.cancel / check_in / reschedule`)

```
reservation = get_object_or_404(Reservation, pk=pk)
try:
    cancel_reservation(reservation, request.user)
except OwnershipError as exc:
    raise PermissionDenied(str(exc)) from exc
```

Por que é explorável: A posse é verificada corretamente antes de qualquer escrita — ``cancel_reservation`, `check_in_reservation`` e ``reschedule_reservation`` levantam ``OwnershipError`` quando ``reservation.user != user`` (`reservations/services.py:149,254,311`), e a view converte isso em ``403``. O ponto é que a busca por ``pk`` acontece sem filtrar por dono: um ``pk`` de reserva alheia responde ``403`` e um ``pk`` inexistente responde ``404`` — a diferença permite a um usuário autenticado enumerar quais IDs de reserva existem no sistema (sem nunca conseguir ler ou alterar o conteúdo de uma reserva alheia).

Condição de exploração: Vazamento apenas de existência de ID sequencial, não de conteúdo — nenhum dado da reserva alheia é exposto. Severidade baixa/informativa; citado porque o padrão mais forte (``get_object_or_404(..., user=request.user)``) já é usado em todas as views web equivalentes (ver Pontos Fortes) e poderia ser replicado aqui por consistência.

4. Chaves expostas

Média

F1 — Guarda de startup contra `SECRET_KEY` insegura cobre só o próprio default

Arquivo: `conf/settings.py:229-234`

```
if not DEBUG:
    if SECRET_KEY.startswith("django-insecure-"):
        raise ImproperlyConfigured(
            "SECRET_KEY não foi definida. Configure a variável de ambiente..."
        )
```

Por que é explorável: A checagem só rejeita a string literal que o próprio Django gera (``django-insecure-...``, `settings.py:29-32`). Ela não rejeita o valor de exemplo distribuído em ``.env.example:2`` (``SECRET_KEY=change-me-in-production``) nem qualquer outro placeholder óbvio. Um operador que copia ``.env.example`` para ``.env``, esquece de trocar o ``SECRET_KEY`` e sobe com ``DEBUG=0`` fica com uma chave de assinatura de sessão/CSRF pública e previsível, sem nenhum aviso do sistema.

Condição de exploração: Requer que o operador realmente esqueça de trocar o valor de exemplo — mas o ``.docker-compose.prod.yml:41`` só exige que a variável ``*exista*`` (``${SECRET_KEY:?...}``), não que ela seja diferente do exemplo. Nenhuma trava impede copiar o ``.env.example`` inteiro.

``.env.example:2``: `SECRET_KEY=change-me-in-production`

Média

F3 — ``seed_data`` cria superusuário 'admin' com senha padrão previsível, sem guarda de ambiente

Arquivo: `core/seeds/users.py:7-34` e `core/management/commands/seed_data.py`

```
DEFAULT_USERS = [
    {"username": "admin", ..., "is_staff": True, "is_superuser": True},
    ...
]
DEFAULT_PASSWORD = os.environ.get("SEED_DEFAULT_PASSWORD", "reserva123")
```

Por que é explorável: O comando ``python manage.py seed_data`` (exposto também como ``make seed``) cria (ou reseta a senha de) um usuário ``admin`` com ``is_staff=True, is_superuser=True`` e a senha padrão ``reserva123`` sempre que ``SEED_DEFAULT_PASSWORD`` não estiver definida. O comando não verifica ``settings.DEBUG`` nem qualquer outra condição de ambiente antes de rodar — nada no código o impede de ser executado contra um banco de produção.

Condição de exploração: Requer que alguém rode ``make seed`` / ``manage.py seed_data`` apontando para um banco real sem sobrescrever ``SEED_DEFAULT_PASSWORD`` — um erro operacional plausível em ambientes de homologação promovidos a produção sem trocar de banco.

Baixa

F2 — Placeholder de senha do Postgres não é validado, só exigido como não-vazio

Arquivo: `.env.example:19` e `docker-compose.prod.yml:20`

```
POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:?defina POSTGRES_PASSWORD no .env}
```

Por que é explorável: O compose de produção corretamente recusa subir sem `POSTGRES_PASSWORD` definida (:?), mas nada impede que o valor definido seja literalmente o placeholder do `.env.example:19` ('troque-esta-senha') — o compose não sabe distinguir 'foi definida' de 'foi trocada'.

Condição de exploração: Mesmo cenário do F1: só é explorável se o operador copiar o `.env.example` sem editar os placeholders antes de subir em produção.

Informativa

F7 — Nenhum hook de detecção de segredos no pre-commit

Arquivo: `pre-commit-config.yml`

```
repos:
- repo: https://github.com/compilerla/conventional-pre-commit
- repo: https://github.com/astral-sh/ruff-pre-commit
- repo: local # pytest
```

Por que é explorável: O pre-commit configurado cobre lint, formatação, mensagem de commit e testes, mas nenhum hook (ex.: `gitLeaks`, `detect-secrets`) varre o conteúdo dos arquivos por segredos antes do commit. Hoje o `.env` está corretamente fora do controle de versão (verificado no histórico do Git), mas essa proteção depende inteiramente do `.gitignore` continuar correto — não há uma segunda camada automatizada.

Condição de exploração: N/A — recomendação de fortalecimento preventivo, não uma falha ativa.

Informativa

F8 — Chave real da API Groq presente no `.env` local, lida durante esta auditoria

Arquivo: `.env` (não versionado)

```
GROQ_API_KEY=gsk_***** (valor real, redigido neste relatório)
```

Por que é explorável: Durante a varredura de defaults inseguros, o `.env` local do ambiente auditado foi lido e continha uma chave de API real da Groq. O arquivo está corretamente listado em `.gitignore:54` e nunca foi commitado (confirmado via `git log --all` e `git ls-files`) — não há evidência de vazamento para o repositório ou para terceiros pelo código. Como o valor passou pela sessão de auditoria (terminal/contexto do assistente), a recomendação é rotacionar essa chave por precaução operacional, não porque uma vulnerabilidade de código a expôs.

Condição de exploração: N/A — não é uma falha de código; é uma recomendação de higiene operacional.

5. Inputs sem tratamento (XSS)

Nenhum achado nesta categoria. Ver seção "Pontos fortes" para as proteções verificadas — nenhum uso de `|safe`, `mark_safe` ou `format_html` sobre conteúdo de usuário/administrador/LLM foi encontrado em todo o código-fonte.

Recomendações priorizadas

Prior.	Recomendação
P1	Endurecer a guarda de startup contra segredos-placeholder. Trocar a checagem de <code>`SECRET_KEY.startswith("django-insecure-")`</code> por uma lista de valores proibidos (incluindo o do <code>`env.example`</code>) ou por uma checagem de entropia mínima; aplicar o mesmo raciocínio a <code>`POSTGRES_PASSWORD`</code> . Referências: F1, F2.
P1	Proteger <code>`seed_data`</code> contra execução em produção. Adicionar uma checagem no início de <code>`Command.handle`</code> (ex.: recusar rodar quando <code>`not settings.DEBUG`</code> sem uma flag explícita <code>`--eu-sei-o-que-estou-fazendo`</code>), e considerar gerar uma senha aleatória por execução em vez de um default fixo. Referência: F3.
P2	Adicionar filtro <code>`is_active=True`</code> ao <code>`SpaceViewSet`</code> (ou expor a exceção deliberadamente). Alinhar <code>`SpaceViewSet.get_queryset`</code> ao comportamento de <code>`SpaceListView`</code> , ou — se o acesso de leitura a espaços inativos por qualquer usuário autenticado for intencional — documentar a decisão explicitamente no código, como já é feito em outros pontos do projeto. Referência: F5.
P2	Decidir e documentar o modelo de privilégio de <code>`is_staff`</code>. Definir se 'qualquer staff pode criar outro staff' é a política desejada; se não for, restringir o campo <code>`is_staff`</code> do formulário/view a <code>`is_superuser`</code> , ou introduzir um segundo nível de permissão. Referência: F4.
P3	Buscar reserva já filtrada por dono nas ações do DRF (<code>`cancel`/`check_in`/`reschedule`</code>). Trocar <code>`get_object_or_404(Reservation, pk=pk)`</code> por <code>`get_object_or_404(Reservation, pk=pk, user=request.user)`</code> nessas três ações, replicando o padrão já usado nas views web, para eliminar o oráculo de existência. Referência: F6.
P3	Adicionar hook de detecção de segredos ao pre-commit. Incluir <code>`gitLeaks`</code> ou <code>`detect-secrets`</code> em <code>`.pre-commit-config.yaml`</code> como segunda camada de proteção, independente do <code>`.gitignore`</code> . Referência: F7.
P3	Rotacionar a chave da Groq usada no ambiente auditado. Por higiene operacional, já que o valor circulou nesta sessão de auditoria — não porque uma falha do código a expôs. Referência: F8.

Issues para o GitHub

Texto completo em Markdown, pronto para copiar e colar na criação de cada issue. Achados triviais e relacionados foram agrupados numa única issue quando fazia sentido, para não gerar spam.

--- ISSUE 1 ---

Título: [Segurança] Defaults de segredo (.env.example) não são rejeitados na inicialização

Labels sugeridas: security, media

```
## Problema

`config/settings.py:229-234` recusa subir com `DEBUG=False` apenas se `SECRET_KEY` começar com o
prefixo `django-insecure-` (o default que o próprio Django gera). Ele não rejeita outros valores de
exemplo conhecidos que já estão no repositório, como `SECRET_KEY=change-me-in-production`
(`.env.example:2`) ou `POSTGRES_PASSWORD=troque-esta-senha` (`.env.example:19`,
usado em `docker-compose.prod.yml:20`).
```

Um operador que copia `.env.example` para `.env`, esquece de trocar esses dois valores e sobe com
`DEBUG=0` conclui o deploy sem nenhum aviso, com a chave de assinatura de sessão/CSRF e a senha do
banco publicamente conhecidas (estão neste mesmo repositório).

```
## Por que é explorável

`docker-compose.prod.yml` usa `${SECRET_KEY:?defina SECRET_KEY no .env}` — isso garante que a
variável *existe*, não que ela foi *trocada*. Não há nenhuma validação de conteúdo.
```

```
## Evidência

`config/settings.py:229-234`
```python
if not DEBUG:
 if SECRET_KEY.startswith("django-insecure-"):
 raise ImproperlyConfigured(
 "SECRET_KEY não foi definida. Configure a variável de ambiente SECRET_KEY "
 "antes de rodar com DEBUG desligado."
)
...

`.env.example:2` — `SECRET_KEY=change-me-in-production`
`.env.example:19` — `POSTGRES_PASSWORD=troque-esta-senha`
```

```
Impacto

Com `SECRET_KEY` conhecida, um atacante pode forjar cookies de sessão assinados e tokens CSRF,
comprometendo a autenticação de qualquer usuário. Com `POSTGRES_PASSWORD` conhecida (e a porta do
banco exposta, como em `docker-compose.yml:12-13`), acesso direto ao banco de produção.
```

```
Sugestão de correção

- Ampliar a checagem de `SECRET_KEY` para uma lista de valores proibidos conhecidos (incluindo o
do `.env.example`), ou trocar por uma checagem de entropia/comprimento mínimo.
- Adicionar checagem equivalente para `POSTGRES_PASSWORD` em algum ponto de inicialização (ex.:
healthcheck do banco, ou script de endpoint do container `db`).
- Considerar gerar `.env.example` com instrução para rodar
`python -c "from django.core.management.utils import get_random_secret_key as k; print(k())"`
como parte de um script `make setup`, em vez de um valor fixo copiável.
```

```
Critérios de aceite

- [] Subir com `DEBUG=0` e `SECRET_KEY=change-me-in-production` falha no startup com mensagem
clara.
- [] Subir com `DEBUG=0` e `POSTGRES_PASSWORD=troque-esta-senha` falha (ou é bloqueado antes do
primeiro request) com mensagem clara.
- [] Teste automatizado cobre os dois casos acima.
```

## --- FIM ISSUE 1 ---

## --- ISSUE 2 ---

**Título:** [Segurança] `seed\_data` cria superusuário com senha padrão previsível, sem guarda de ambiente

**Labels sugeridas:** security, media

```
Problema

`python manage.py seed_data` (também `make seed`) cria/atualiza um usuário `admin` com
`is_staff=True, is_superuser=True` e senha `reserva123` sempre que a variável de ambiente
`SEED_DEFAULT_PASSWORD` não estiver definida. O comando não verifica `settings.DEBUG` nem nenhuma
outra condição antes de rodar.

Por que é explorável

Se alguém rodar esse comando contra um banco de produção (por engano, ou porque um ambiente de
homologação foi promovido a produção sem trocar de banco), o sistema passa a ter uma conta
administrativa com credencial pública e documentada no próprio repositório.

Evidência

`core/seeds/users.py:7-34`
```python
DEFAULT_USERS = [
    {"username": "admin", "first_name": "Administrador", "is_staff": True, "is_superuser": True},
    ...
]
DEFAULT_PASSWORD = os.environ.get("SEED_DEFAULT_PASSWORD", "reserva123")
```

`core/management/commands/seed_data.py` – `handle()` não checa ambiente antes de chamar
`users.seed()`.

Impacto

Comprometimento total do sistema (conta com `is_superuser=True`) caso o comando seja executado
contra um banco real sem `SEED_DEFAULT_PASSWORD` customizada.

Sugestão de correção

- Recusar rodar quando `not settings.DEBUG`, exigindo uma flag explícita de confirmação
 (`--eu-sei-o-que-estou-fazendo` ou similar) para o caso raro de precisar rodar em produção.
- Gerar uma senha aleatória por execução e imprimi-la uma única vez no terminal, em vez de um
 default fixo.

Critérios de aceite

- [] `manage.py seed_data` sem flag de confirmação falha quando `DEBUG=False`.
- [] Teste automatizado cobre esse bloqueio.
```

## --- FIM ISSUE 2 ---

### --- ISSUE 3 ---

**Título:** [Segurança] API de espaços (DRF) expõe espaços inativos que a tela web esconde

**Labels sugeridas:** security, baixa

```
Problema

`SpaceListView` (a tela web) filtra `is_active=True`. `SpaceViewSet` (`/api/v1/spaces/`, DRF) não aplica esse filtro por padrão e ainda expõe `is_active` como filtro exato via `SpaceFilterSet`, permitindo `GET /api/v1/spaces/?is_active=false` para qualquer usuário autenticado.

Por que é explorável

Basta estar autenticado (não requer papel administrativo). Um espaço desativado por motivo sensível (interdição, obra, uso reservado) fica visível via API mesmo estando escondido na tela.

Evidência

`spaces/views.py:47-51`
```python
class Meta:
    model = Space
    fields = ["is_active", "location", "space_type"]
...

`spaces/views.py:63-69`
```python
class SpaceViewSet(viewsets.ModelViewSet):
 queryset = Space.objects.select_related("space_type").prefetch_related(
 "space_attributes__attribute"
)
...

Comparar com `spaces/views.py:106`: `Space.objects.filter(is_active=True)`.

Impacto

Divulgação de informação de baixo risco (existência/detalhes de espaços que a administração decidiu esconder da tela principal).

Sugestão de correção

Alinhar o queryset padrão do `SpaceViewSet` ao da tela web (`is_active=True`), ou, se o acesso for intencional, documentar a decisão no código.

Critérios de aceite

- [] `GET /api/v1/spaces/` não retorna espaços com `is_active=False` para usuários não-admin.
- [] Teste automatizado cobre o caso.
```

--- FIM ISSUE 3 ---

### --- ISSUE 4 ---

**Título:** [Segurança] Ações de reserva no DRF (cancel/check-in/reschedule) permitem enumerar IDs de reservas alheias

**Labels sugeridas:** security, baixa

```
Problema

`ReservationViewSet.cancel`, `.check_in` e `.reschedule` buscam a reserva só por `pk`
(`get_object_or_404(Reservation, pk=pk)`), sem filtrar por dono. A posse é verificada em seguida
pela camada de serviço (`OwnershipError` → HTTP 403), então nenhum dado da reserva alheia é
exposto – mas a diferença entre 403 (existe, não é sua) e 404 (não existe) permite enumerar IDs de
reserva válidos.

Por que é explorável

Qualquer usuário autenticado pode fazer `PATCH /api/v1/reservations/<pk>/cancel/` variando `<pk>` e
observar o código de status para descobrir quais IDs existem no sistema.

Evidência

`reservations/views.py:110-134`
```python
reservation = get_object_or_404(Reservation, pk=pk)
try:
    cancel_reservation(reservation, request.user)
except OwnershipError as exc:
    raise PermissionDenied(str(exc)) from exc
```

Comparar com o padrão já usado nas views web equivalentes, ex. `reservations/views.py:1158`:
`get_object_or_404(Reservation, pk=pk, user=request.user)`.

Impacto

Vazamento de existência de ID sequencial apenas – nenhum conteúdo de reserva alheia é lido ou
alterado. Severidade baixa.

Sugestão de correção

Trocar `get_object_or_404(Reservation, pk=pk)` por
`get_object_or_404(Reservation, pk=pk, user=request.user)` nas três ações, replicando o padrão já
usado nas views web (elimina o oráculo e simplifica o código, já que `OwnershipError` deixa de ser
alcançável por essas rotas).

Critérios de aceite

- [] `PATCH /api/v1/reservations/<pk-de-outro-usuario>/cancel/` responde 404, não 403.
- [] O mesmo para `check-in` e `reschedule`.
- [] Teste automatizado cobre os três casos.
```

--- FIM ISSUE 4 ---

### --- ISSUE 5 ---

**Título:** [Segurança] Fortalecimento preventivo: hook de detecção de segredos e rotação de chave de sessão

**Labels sugeridas:** security, informativa

```
Problema

Dois itens de fortalecimento preventivo, sem falha ativa associada:

1. `.pre-commit-config.yaml` não tem nenhum hook de detecção de segredos (gitleaks/detect-secrets).
 Hoje `.env` está corretamente fora do controle de versão (confirmado via `git log --all` e
 `git ls-files`), mas essa proteção depende inteiramente do `.gitignore` continuar correto, sem
 uma segunda camada automatizada.
2. Durante a auditoria de segurança, o `.env` local do ambiente contendo uma chave real da API
 Groq foi lido para verificar ausência de segredos versionados. O valor nunca chegou ao
 repositório, mas circulou pela sessão de auditoria – recomenda-se rotacionar essa chave por
 precaução operacional.

Por que agrupar numa issue só

Nenhum dos dois é uma vulnerabilidade explorável hoje; são recomendações de higiene que não
justificam issues separadas.

Sugestão de correção

- Adicionar `gitleaks` (ou `detect-secrets`) a `.pre-commit-config.yaml`.
- Rotacionar a `GROQ_API_KEY` do ambiente auditado no painel da Groq.

Critérios de aceite

- [] `.pre-commit-config.yaml` roda um hook de detecção de segredos em todo commit.
- [] `GROQ_API_KEY` do ambiente auditado foi rotacionada.
```

--- FIM ISSUE 5 ---